A Multi-Granularity Retrieval Augmented Generation Framework for EDA Tasks Leveraging Knowledge Graph

Yuheng Huang
Beijing University of Posts and Telecommunications
Beijing, China

Yuhan Qin
The Chinese University of Hong Kong
Hong Kong, China

Yuan Pu
The Chinese University of Hong Kong
Hong Kong, China

Zhaotan Lin
The Chinese University of Hong Kong
Hong Kong, China

Yuntao Nie
Beijing University of Posts and Telecommunications
Beijing, China

Zhuolun He*
The Chinese University of Hong Kong
Hong Kong, China

Jianwang Zhai†
Beijing University of Posts and Telecommunications
Beijing, China

Kang Zhao
Beijing University of Posts and Telecommunications
Beijing, China

Bei Yu
The Chinese University of Hong Kong
Hong Kong, China

Abstract

Retrieval-augmented generation (RAG) supports knowledge-intensive tasks such as question answering (QA) and code generation over domain-specific documentation. Because this documentation encodes structured knowledge, chunk-based approaches face a granularity dilemma: coarse chunks introduce irrelevant context, while fine chunks fragment relational and structural information. We therefore construct a knowledge graph (KG) and index its knowledge at four granularities, from property- and node-level facts to edge- and subgraph-level relations. Rather than merging all granularities into one retrieval space, our structure-aware algorithm localizes the query-relevant KG region and ranks cross-granularity candidates within it. Experiments on EDA-domain QA and code generation benchmarks show consistent improvements over existing baselines on both tasks; the code is available at https://github.com/meaqual/MultiGranKG.

CCS Concepts

• **Hardware** → **Electronic design automation**; • **Information systems** → *Retrieval models and ranking*.

Keywords

Retrieval-Augmented Generation, Knowledge Graph, Electronic Design Automation, Code Generation, Question Answering

ACM Reference Format:
Yuheng Huang, Yuhan Qin, Yuan Pu, Zhaotan Lin, Yuntao Nie, Zhuolun He, Jianwang Zhai, Kang Zhao, and Bei Yu. 2026. A Multi-Granularity Retrieval Augmented Generation Framework for EDA Tasks Leveraging Knowledge Graph . In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '26), November 08–12, 2026, San Jose, CA, USA.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3831252.3834019

*Also with Tsinghua University.
†Corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.

ICCAD '26, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2873-0/2026/11
https://doi.org/10.1145/3831252.3834019

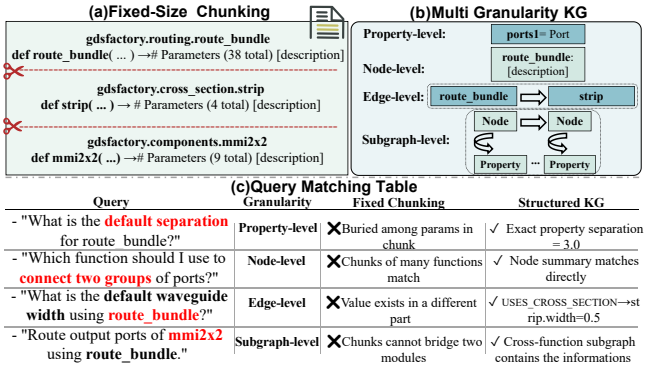


Figure 1: Motivating example for granularity-aware retrieval in EDA tool documentation.

1 Introduction

LLM-based assistants increasingly support EDA tasks such as documentation QA and code generation [5, 12, 15, 27, 28, 39]. Their primary knowledge sources include manuals, API references, parameter descriptions, and procedural guides. Unlike general technical text, however, these documents encode tightly coupled command-parameter semantics, cross-referenced API objects, and ordered procedures. Relevant evidence may therefore range from one local fact to multiple dependent documentation elements, making effective retrieval difficult.

Retrieval-augmented generation (RAG) commonly grounds such systems in external documentation [1, 10, 18, 20–22, 29, 32, 33]. Standard RAG segments documents into fixed-size chunks and supplies the top-ranked chunks for answer or code generation [19, 38, 42, 43]. This assumes that evidence is self-contained at a fixed scale, whereas EDA queries require different scopes. Coarse chunks add irrelevant context, while fine chunks separate dependencies needed for reliable generation.

Retrieval scope is therefore query-dependent: a parameter lookup may need one property, whereas API use or troubleshooting may require relations among several objects and steps. Effective retrieval must preserve both scopes without imposing a fixed context size,

balancing local precision with relational completeness at query time.

Existing work does not fully resolve this mismatch. EDA-oriented systems cover documentation QA, code generation, and agent-based tool interaction [12–14, 24, 26–28, 36, 37, 39], but their retrieval components generally do not adapt evidence granularity to each query. Graph- and knowledge-aware methods incorporate structure through graph reasoning, iterative retrieval, or graph-organized summaries [7–9, 11, 23, 25, 34, 35, 40, 41], yet commonly rely on one dominant corpus representation. They thus do not explicitly coordinate the atomic properties, relations, and procedural fragments required by different EDA queries, as illustrated in Fig. 1(a) and (c).

We address this limitation by representing EDA documentation as a knowledge graph and organizing its entities, properties, relations, and coordinated structures at four complementary granularities. As shown in Fig. 1(b) and (c), different queries can then access different levels of detail rather than one fixed partition. The formulation supports different documentation families through corpus-specific object and relation types under the same structured retrieval framework. Because globally mixing all granularities creates redundant views and ranking interference, our pipeline first localizes the query-relevant graph region and then ranks cross-granularity candidates within it.

We evaluate the framework on documentation QA and code generation, observing consistent gains driven by coordinated retrieval across levels of detail rather than merely exposing more granularities. Our main contributions are as follows:

- We identify the granularity mismatch in EDA documentation: different queries require evidence at different scopes, making fixed-size retrieval inadequate.
- We propose a retrieval framework for EDA tool documentation based on a multi-granularity knowledge graph. It organizes document knowledge at four complementary granularities and implements a retrieval pipeline supporting reliable cross-granularity retrieval without merging all granularities into a single space.
- We conduct retrieval-conditioned evaluation on QA and code generation over EDA tool documentation, and show that the proposed framework consistently outperforms strong baselines on both retrieval and end-to-end performance, improving Doc QA JudgeAcc from 0.717 to 0.807 over the strongest compared retrieval baseline and improving Combined CodeGen API F1 by 0.239.

2 Preliminaries

2.1 Retrieval-Conditioned EDA Tasks

Retrieval-conditioned QA and code generation over EDA tool documentation both require relevant documentation evidence before generation: QA grounds natural-language answers in command, parameter, API, or procedural semantics, whereas code generation requires complete API relations and workflow context to produce valid code [24, 36–38, 42, 43]. Given a query q and documentation corpus $\mathcal{D}$, retrieval selects an evidence set $C(q) \subset \mathcal{D}$ for a generator. Success therefore depends on both evidence coverage and the generator's use of that evidence; fluent output cannot compensate for a missing constraint, relation, or procedural step.

2.2 Retrieval-Augmented Generation

RAG retrieves external evidence as context for an LLM [1, 10, 20, 22, 29, 33]. Standard pipelines segment documents into chunks, retrieve them with dense or late-interaction models, and optionally rerank candidates [3, 18, 21, 32]. Their effectiveness in documentation QA and code generation depends on evidence that is both relevant and sufficiently complete [19, 38, 42, 43]. Retrieval is thus evaluated independently through evidence recall and jointly through downstream correctness, distinguishing retrieval failure from generation failure.

Recall measures whether required evidence reaches the generator, whereas downstream metrics measure whether it supports a correct answer or API-aligned program. Reporting both separates retrieval and generation errors.

2.3 Graph-Based Retrieval

Graph-based retrieval represents corpus entities and relations explicitly, enabling connectivity-based expansion and structure-aware ranking beyond independent text passages [7–9, 11, 23, 25, 34, 35, 40, 41]. This is useful when evidence depends on both local semantics and relations. However, graph connectivity alone does not determine whether a query needs an attribute, a relation, or a larger coordinated structure; retrieval units and ranking must also account for evidence scope and expose multiple views of the same structured knowledge.

Our method represents API signatures, parameters, return types, constraints, and workflows as schema-validated objects. Unlike textual multi-grained methods such as KET-RAG [17] and our IMG-RAG adaptation of MGranRAG [16], it explicitly encodes EDA relations and retrieves them through relation anchors, typed expansion, cross-granularity ranking, and coverage-aware selection.

3 Method

3.1 Framework Overview

The proposed framework addresses retrieval over EDA tool documentation through two tightly connected parts: offline knowledge organization and online retrieval. The central challenge is that relevant evidence may appear at different levels of detail, while naively merging all granularities into a single retrieval space introduces redundant cross-granularity views of the same underlying knowledge and makes ranking less reliable. Offline, the framework converts parsed EDA tool documentation into a schema-guided knowledge graph and derives retrieval indexes at four complementary granularities: property-level, node-level, edge-level, and subgraph-level. Although the concrete schema is instantiated separately for different corpora, all instantiations follow the same typed formulation with node types, edge types, hyperedge types, associated fields, and type constraints. Online, retrieval proceeds in three stages: graph localization, cross-granularity ranking, and coverage-aware evidence selection, using the structured retrieval space built offline to retrieve evidence at the level of detail required by each query.

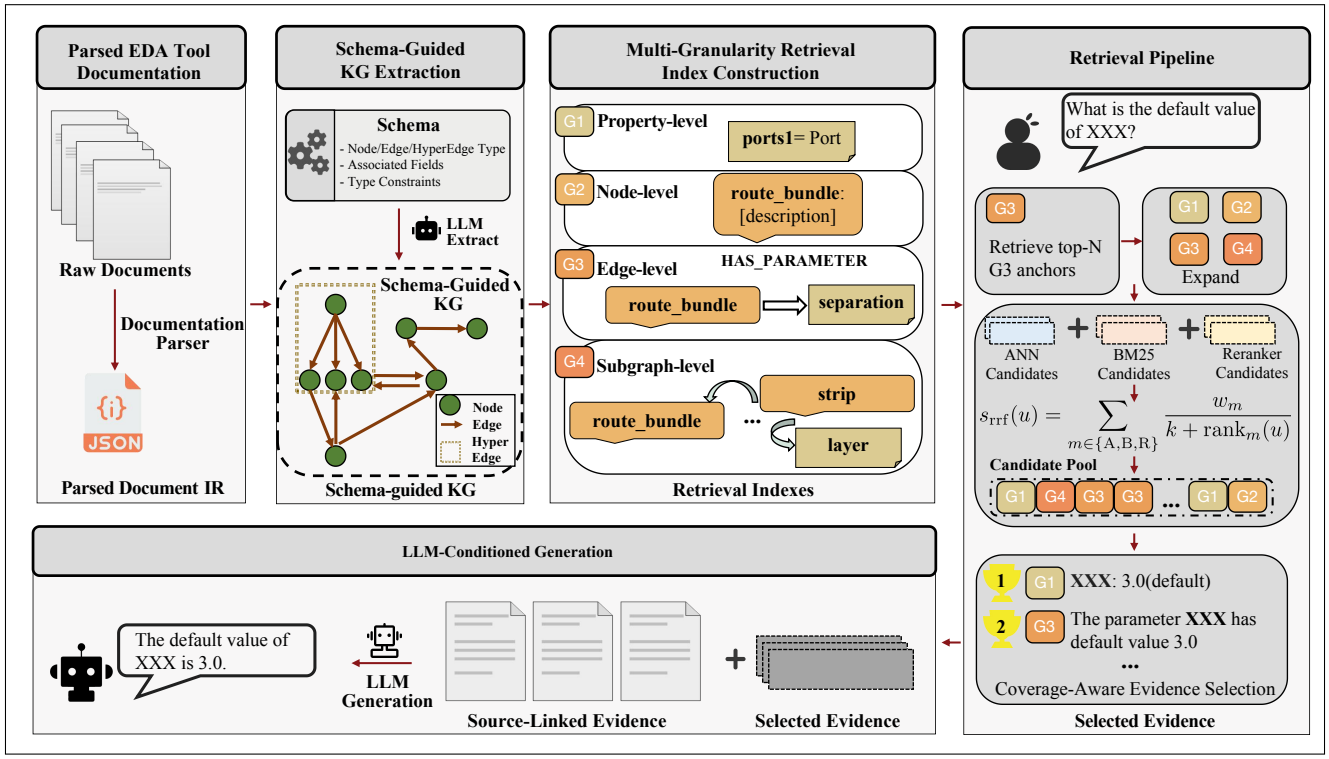


Figure 2: Overview of the proposed framework. The retrieval indexes correspond to four granularities: property-level (G1), node-level (G2), edge-level (G3), and subgraph-level (G4). It first organizes parsed EDA tool documents into a schema-guided knowledge graph and retrieval indexes, then applies the retrieval pipeline for downstream generation.

3.2 Schema-Guided KG Construction

The first challenge is to convert heterogeneous EDA tool documentation into a graph representation that is both structurally faithful and retrievable. EDA documentation is not organized uniformly: some corpora center on commands, options, and procedures, while others center on modules, classes, methods, and parameters. Without schema constraints, extraction would produce inconsistent object types, noisy relations, and fragmented higher-order structures. We therefore adopt a schema-guided formulation and perform graph extraction under an explicit typed representation space.

As shown in Fig. 2, KG extraction follows three constrained prompts: entity identification, entity enrichment, edge and hyperedge extraction; each exposes the valid types and requires source-linked JSON. The parser normalizes outputs, filters invalid types, checks relation endpoint types and source grounding, and removes unsupported objects. Nodes are consolidated by entity type and normalized name, edges by relation type and endpoints, and hyperedges by type, participating nodes, and logic; aliases, attributes, and source references are merged. For example, the GDSFactory schema restricts `HAS_PARAMETER` to connect functions or methods with parameters and RETURNS to connect callable APIs with data types.

Every retained graph element preserves its originating documentation unit and extraction evidence. Consequently, validation failures can be rejected without breaking provenance, while consolidated aliases and repeated mentions remain traceable to their supporting sources. This traceability later permits retrieved graph records to be mapped back to generator-ready documentation evidence.

3.3 Multi-Granularity Retrieval Index Construction

Although structurally expressive, the knowledge graph is not directly suitable for dense, lexical, and reranking-based retrieval, which require serializable records comparable across evidence scopes. We therefore derive four retrieval indexes from the schema-guided graph, preserving its structure while exposing retrievable text at complementary levels of detail.

Each retrieval index is constructed offline from graph elements and inherits their structural traceability. Every indexed record includes a textual serialization, a granularity label, a covered graph-element key set, and source links inherited from the graph. As illustrated in Fig. 3, the property-level granularity index (G1) stores records from individual attributes of graph objects, the node-level granularity index (G2) stores one graph object together with its selected key attributes, the edge-level granularity index (G3) stores explicit typed relations, and the subgraph-level granularity index (G4) stores connected local graph structures that preserve coordinated evidence such as signatures, parameter groups, workflow

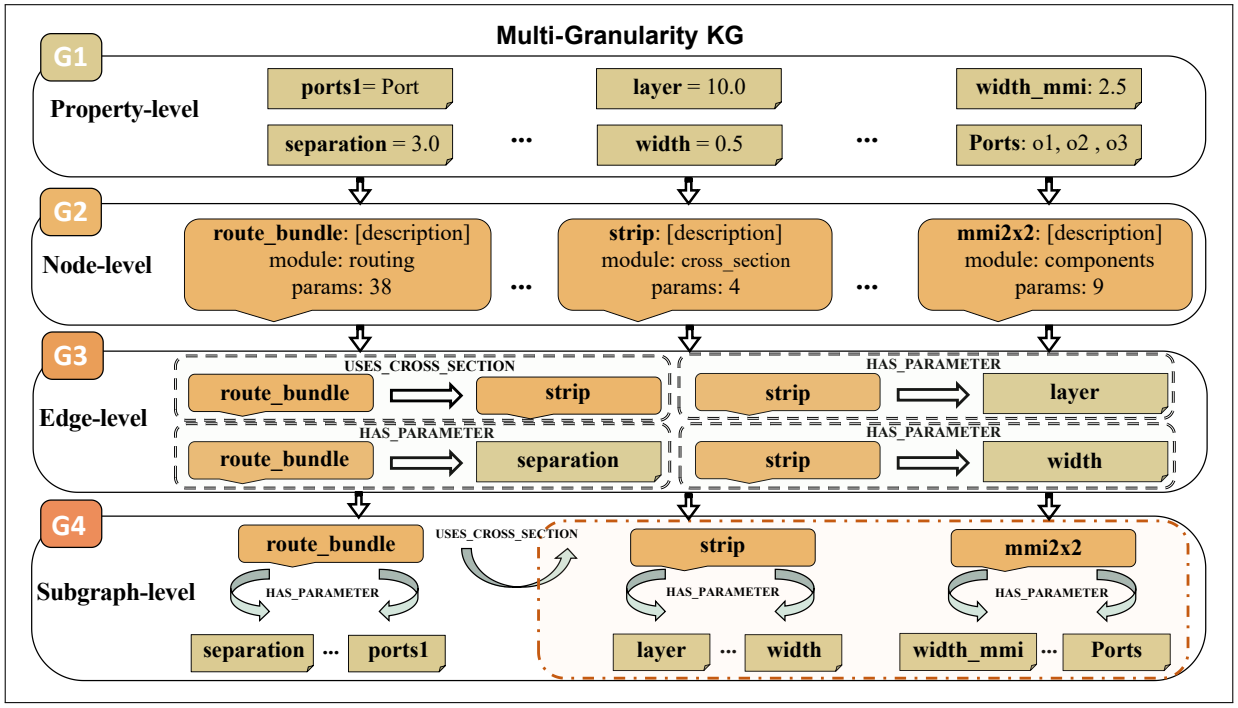


Figure 3: Illustration of the multi-granularity knowledge graph and the corresponding retrieval indexes. G1, G2, G3, and G4 denote the four index granularities.

fragments, troubleshooting chains, and module overviews. The serialization supports direct indexing and retrieval, while the retained key sets and source links preserve alignment with the underlying graph structure and supporting document evidence.

After construction, all four indexes are serialized and embedded offline. We also build a reverse map from graph elements to indexed records, together with a bidirectional graph adjacency structure for later expansion and neighborhood access. This keeps graph structure, retrievable text, and source traceability aligned, and also makes G3 a natural anchor for later localization: compared with G1 and G2, it retains explicit relational cues, while compared with G4, it remains sufficiently local for precise expansion.

The index partition is fixed offline; query-dependent granularity weighting is realized online through graph localization, cross-granularity ranking, and coverage-aware evidence selection.

3.4 Retrieval Pipeline

Given a query q, the online retrieval stage should not compare all candidates in one global mixed space, because redundant cross-granularity views of the same knowledge can make ranking less reliable. Instead, the proposed pipeline performs retrieval in three stages: graph localization, cross-granularity ranking, and coverage-aware evidence selection. Once documentation knowledge has been organized into typed graph elements and exposed through retrieval indexes at multiple granularities, the same online formulation can be applied across different EDA documentation families.

Query-Relevant Graph Localization. Graph localization begins by retrieving the top-N edge-level (G3) candidates according to dense similarity between the query embedding and the G3 index. G3 is used as the anchor because it preserves explicit relational information while remaining sufficiently local for reliable expansion. The graph elements covered by these G3 candidates are then used as seeds for multi-hop expansion on the graph, producing a localized graph region $\mathcal{G}_{\text{loc}}(q)$. Using the reverse map from graph elements to indexed records, the method gathers all G1, G2, G3, and G4 candidates whose covered graph elements intersect $\mathcal{G}_{\text{loc}}(q)$, yielding a graph-local candidate pool. To avoid missing relevant evidence outside this localized region, the pipeline also retrieves an ANN pool independently from each of the four indexes. After merging and deduplication, the final candidate pool remains compact while preserving recall across granularities.

Cross-Granularity Ranking. After graph localization, the remaining problem is how to compare candidates drawn from different granularities in a consistent way. A single retrieval score is often unreliable here because candidates at different granularities differ substantially in scope, length, and redundancy. The proposed method therefore performs cross-granularity ranking within the localized candidate pool rather than ranking all candidates directly in one global mixed space.

Let P denote the localized candidate pool, and let each candidate $u \in P$ be associated with a covered graph element key set $\mathcal{E}(u)$. The ranking stage combines three complementary signals: dense similarity, BM25 lexical relevance [31], and reranker score. The reranker is applied only after a lightweight prefilter, so that only the most promising candidates are rescored. These three rankings

are then fused by reciprocal-rank fusion (RRF) [6]:

$$s_{\mathrm{rrf}}(u) = \sum_{m \in \{\mathrm{ann,bm25,rerank}\}} \frac{w_m}{k + \mathrm{rank}_m(u)}. \tag{1}$$

The fused score is refined by two structure-aware corrections. The first is a generic penalty for overly broad G4 candidates, which discourages high ranking solely from broad but weakly focused coverage. The second is graph element support, which promotes candidates whose covered graph elements are repeatedly supported by other high-scoring candidates in the same localized pool.

We define the support weight of graph element key e as

$$\omega(e) = \mathrm{idf}(e) \sum_{u \in P:\, e \in \mathcal{E}(u)} \frac{s_{\mathrm{rrf}}(u)}{|\mathcal{E}(u)|}, \tag{2}$$

where

$$\mathrm{idf}(e) = \log \frac{|P| + 1}{\mathrm{df}(e) + 1}, \qquad \mathrm{df}(e) = |\{u \in P :\, e \in \mathcal{E}(u)\}|. \tag{3}$$

A graph element therefore receives a larger weight when it is repeatedly supported by high-scoring candidates, while elements that appear too broadly across the candidate pool are downweighted. The graph element support score of candidate u is then

$$s_{\mathrm{elem}}(u) = \frac{1}{|\mathcal{E}(u)|} \sum_{e \in \mathcal{E}(u)} \omega(e). \tag{4}$$

The final ranking score is

$$s(u) = s_{\mathrm{rrf}}(u) - \gamma\, p_{\mathrm{gen}}(u) + \eta\, s_{\mathrm{elem}}(u), \tag{5}$$

where $p_{\mathrm{gen}}(u)$ denotes the generic penalty, and γ and η control the strengths of the two corrections.

This stage differs from naively merging all granularities into a single retrieval space and ranking them globally. The pipeline first restricts comparison to a query-relevant localized region and then ranks candidates there using both retrieval signals and graph-based structural support, which makes cross-granularity comparison more stable and reduces interference from unrelated or redundant candidates.

Coverage-Aware Evidence Selection. After ranking, the remaining problem is no longer candidate relevance, but candidate overlap. Many high-scoring candidates cover largely the same graph elements, especially when evidence from different granularities refers to the same underlying knowledge. A pure top-K truncation would therefore over-select redundant evidence and under-cover complementary structural evidence. To address this, our pipeline performs coverage-aware evidence selection with weighted MMR [2].

Let $s(u)$ denote the final ranking score of candidate u, and let $\mathcal{E}(u)$ denote the set of graph element keys covered by u. During greedy selection, the method maintains the set C of graph elements already covered by the selected evidence. For each remaining candidate, it computes the uncovered set

$$\Delta(u) = \mathcal{E}(u) \setminus C \tag{6}$$

and defines a coverage gain

$$\mathrm{gain}(u) = \sum_{e \in \Delta(u)} \omega(e). \tag{7}$$

This gain term rewards candidates that introduce new graph elements with high structural support, rather than candidates that merely repeat evidence already covered by higher-ranked items. The final selection value is

$$\mathrm{val}(u) = s(u) + \lambda\, \mathrm{gain}(u), \tag{8}$$

where λ controls the relevance-coverage tradeoff.

This formulation differs from conventional text-level MMR [2] in two ways. First, diversity is computed over graph coverage rather than pairwise text dissimilarity. Second, the gain term is weighted by structural support, so newly covered graph elements do not contribute equally. The final evidence set is therefore selected jointly by relevance and incremental structural coverage.

3.5 Retrieval-Conditioned Generation

The selected records are mapped through their source links to supporting documentation and serialized as the generator context. Their retrieval order is retained so that the most relevant evidence appears first, while the coverage-aware selection stage limits repeated views of the same graph elements. This produces one common evidence interface for downstream generation without coupling the retrieval pipeline to a task-specific output format.

For QA, the prompt asks the generator to answer from the supplied documentation evidence, which grounds factual claims and procedural explanations in retrieved sources. For code generation, the context exposes the relevant API objects, signatures, parameters, relations, and usage constraints before synthesis, helping the generator select and compose the required interfaces. In both tasks, generation remains separate from retrieval: the graph determines which evidence is supplied, while the LLM produces the task-specific natural-language answer or code. This modular design also permits the generator to be replaced without reconstructing the KG or retrieval indexes.

4 Experimental Results

4.1 Experimental Setup

We evaluate the proposed framework on two retrieval-conditioned tasks over EDA tool documentation: QA and code generation. Unless otherwise specified, all methods use the same BGE-M3 embedder for dense retrieval, the same BGE-based reranker, and the same backbone LLM, prompt template, and decoding settings for fair comparison.

Our evaluation includes both proprietary and public documentation settings. The proprietary benchmark is built from commercial EDA tool documentation and evaluated under Doc QA and Doc CodeGen. To complement this industrial corpus, we construct GDS CodeGen from the GDSFactory documentation corpus. Doc QA contains 522 queries, while code generation contains 222 queries, including 102 in Doc CodeGen and 120 in GDS CodeGen; we also report their union as Combined CodeGen. All benchmarks are annotated and verified by domain experts, and each query is paired with a ground-truth answer, target entities, and aligned evidence spans. Together, they cover two EDA documentation families: tool-oriented documentation centered on commands, parameters, and procedures, and API-oriented documentation centered on modules, classes, functions, and typed signatures.

We compare against seven retrieval baselines and one no-retrieval baseline. NaiveRAG is a dense chunk baseline using 1,024-character

chunks and BGE-M3 embeddings [3]. RAG-EDA is a reproduced dense retrieval baseline with reranking [27]. LightRAG [8], GraphRAG [7], and HippoRAG [9] are representative graph-based baselines. We additionally adapt the textual multi-grained KET-RAG [17] and MGranRAG (denoted IMG-RAG) [16] to the EDA corpora, while LLM-based is a no-retrieval baseline. Each baseline keeps its own retrieval representation, but all methods are evaluated under the same retrieval and generation protocol. For unified retrieval evaluation, we map the top-K evidence returned by each method to the same benchmark target-entity set and compute entity-level Recall@K.

Hyperparameters were fixed before final evaluation using engineering criteria rather than selected on the test set. One-factor sweeps show stable R@20 ranges of 0.891–0.913, 0.901–0.917, and 0.907–0.919 on Doc QA, Doc CodeGen, and GDS CodeGen, with the largest observed shift below 0.019. KG construction is performed once: retrieval artifacts for the three benchmarks require 29.75, 47.36, and 7.87 MB; offline builds take 709.72, 737.64, and 240.16 s with 16.84, 17.83, and 16.36 GB peak memory. Online retrieval and reranking take approximately 1.17, 2.53, and 0.79 s per query.

We report both retrieval and end-to-end generation results. For retrieval, let $T(q)$ denote the set of target entities associated with query q, and let $M_K(q)$ denote the subset covered by the top-K retrieved evidence. We define

$$\text{Recall@K}(q) = \frac{|M_K(q)|}{|T(q)|}, \tag{9}$$

and report Recall@5, Recall@10, and Recall@20.

For QA, we report JudgeAcc and LenientAcc. JudgeAcc is the primary QA metric. LenientAcc counts answers judged either fully correct or materially on target.

For code generation, let $A^*(q)$ denote the normalized set of reference APIs required by query q, and let $\hat{A}(q)$ denote the normalized API set extracted from the generated code. We define

$$\text{API Precision}(q) = \frac{|A^*(q) \cap \hat{A}(q)|}{|\hat{A}(q)|}, \tag{10}$$

$$\text{API Recall}(q) = \frac{|A^*(q) \cap \hat{A}(q)|}{|A^*(q)|}, \tag{11}$$

$$\text{API F1}(q) = \frac{2 \cdot \text{API Precision}(q) \cdot \text{API Recall}(q)}{\text{API Precision}(q) + \text{API Recall}(q)}, \tag{12}$$

$$\text{API Hit}(q) = \mathbf{1}\left[A^*(q) \subseteq \hat{A}(q)\right]. \tag{13}$$

At the code level, we report CodeBLEU [30], defined as

$$\text{CodeBLEU} = \alpha\,\text{BLEU}_{ng} + \beta\,\text{BLEU}_{w} + \gamma\,\text{Match}_{syn} + \delta\,\text{Match}_{df}. \tag{14}$$

These metrics evaluate retrieval quality, QA correctness, API grounding, and code-level similarity, complementing prior code-generation evaluation such as HumanEval [4].

4.2 Main Results

Table 1 shows that our method performs best on every Doc QA metric. Its R@20 reaches 0.909, compared with 0.871 for the strongest retrieval baseline, HippoRAG. The improvement also carries to answer quality: JudgeAcc increases from 0.717 for RAG-EDA, the strongest baseline on this metric, to 0.807. This agreement between retrieval and generation metrics indicates that the recovered graph evidence is useful to the generator rather than only increasing entity coverage.

Table 1: Main results on Doc QA task.

Method	R@5	R@10	R@20	JudgeAcc	LenientAcc
LLM-based(Qwen3 32B)	–	–	–	0.280	0.408
NaiveRAG [3]	0.587	0.685	0.769	0.644	0.776
RAG-EDA [27]	0.745	0.801	0.837	0.717	0.837
LightRAG [8]	0.465	0.496	0.531	0.326	0.490
GraphRAG [7]	0.520	0.585	0.621	0.648	0.803
HippoRAG [9]	0.723	0.805	0.871	0.680	0.768
KET-RAG [17]	0.504	0.601	0.690	0.493	0.647
IMG-RAG [16]	0.595	0.718	0.792	0.608	0.715
Ours	**0.813**	**0.874**	**0.909**	**0.807**	**0.851**

Table 2 shows complementary behavior across the code-generation settings. On Doc CodeGen, HippoRAG has a slightly higher R@20, but our method leads at R@5 and R@10 and substantially improves all three generation metrics, indicating that earlier retrieval of focused evidence matters more than one isolated high-cutoff gain. On GDS CodeGen, our method attains the best retrieval and API F1, while KET-RAG and IMG-RAG remain competitive on CodeBLEU and API Hit. On the combined benchmark, our method achieves the best reported retrieval results and all generation metrics, including an API F1 of 0.557 versus 0.333 for the strongest baseline.

Additional EDA-adapted comparisons support the same conclusion. KET-RAG and IMG-RAG respectively reach R@20 of 0.690 and 0.792 on Doc QA, 0.680 and 0.846 on Doc CodeGen, and 0.801 and 0.887 on GDS CodeGen; ours reaches 0.909, 0.901, and 0.916. A BM25+dense+reranker baseline reaches 0.661, 0.842, and 0.818, showing that conventional hybrid retrieval and reranking alone do not explain the gain. Our method also improves Doc manual correctness from 0.241 to 0.422 and GDS pass rate from 0.631 to 0.675 over this hybrid baseline.

4.3 Why Fixed-Granularity Retrieval Fails

Fixed-granularity retrieval is insufficient for EDA tool documentation, because different queries may depend on localized facts, explicit relations, or broader coordinated structure. We therefore compare four types of retrieval settings: single-granularity retrieval, naive mixed retrieval, the proposed pipeline, and a per-query best-single reference.

Table 3 compares direct retrieval from each G1–G4 index without our pipeline. Mixed exposes all indexes in one shared retrieval space, while Best-Single is a per-query oracle selecting the best G1–G4 result after evaluation.

The results show that neither a single fixed granularity nor naive mixed retrieval is sufficient. On Doc, the strongest single granularity at Recall@20 is G3 with 0.7447, and naive mixed retrieval improves only slightly to 0.7568. Both remain well below the proposed pipeline at 0.9094 and also below the oracle best-single reference at 0.8915. On GDS, the contrast is even clearer: the best single granularity is again G3 with 0.4599, while naive mixed retrieval drops to 0.3885, again far below the proposed pipeline at 0.9156.

Table 2: Main results on code generation benchmarks.

Method	Doc CodeGen				GDS CodeGen				Combined CodeGen			
	R@5/10/20	F1	Hit	BLEU	R@5/10/20	F1	Hit	BLEU	R@5/10/20	F1	Hit	BLEU
NaiveRAG [3]	0.544/0.676/0.783	0.164	0.177	0.212	0.699/0.741/0.756	0.372	0.558	0.765	0.628/0.711/0.769	0.277	0.383	0.511
RAG-EDA [27]	0.720/0.793/0.823	0.159	0.206	0.228	0.742/0.837/0.915	0.374	0.617	0.738	0.732/0.817/0.873	0.275	0.428	0.503
LightRAG [8]	0.350/0.395/0.454	0.163	0.186	0.265	0.813/0.849/0.849	0.239	0.367	0.560	0.600/0.640/0.668	0.204	0.284	0.424
GraphRAG [7]	0.426/0.474/0.516	0.201	0.216	0.258	0.751/0.845/0.894	0.417	0.692	0.703	0.602/0.674/0.721	0.318	0.473	0.499
HippoRAG [9]	0.751/0.828/**0.918**	0.050	0.059	0.166	0.618/0.643/0.675	0.361	0.550	0.739	0.679/0.728/0.787	0.218	0.324	0.476
KET-RAG [17]	0.466/0.578/0.680	0.176	0.196	0.232	0.555/0.665/0.801	0.449	0.733	**0.782**	0.514/0.625/0.745	0.324	0.486	0.529
IMG-RAG [16]	0.668/0.791/0.846	0.185	0.176	0.240	0.778/0.854/0.887	0.458	**0.775**	0.775	0.727/0.825/0.868	0.333	0.500	0.529
Ours	**0.830/0.878**/0.901	**0.350**	**0.363**	**0.275**	**0.885/0.903/0.916**	**0.732**	0.675	0.772	**0.860/0.892/0.909**	**0.557**	**0.532**	**0.544**

Table 3: Single-granularity retrieval, naive mixed retrieval, per-query oracle best-single reference, and the proposed pipeline.

Granularity	Doc			GDS		
	R@5	R@10	R@20	R@5	R@10	R@20
Property-level (G1)	0.422	0.487	0.561	0.274	0.293	0.323
Node-level (G2)	0.461	0.552	0.623	0.310	0.355	0.384
Edge-level (G3)	0.572	0.661	0.745	0.353	0.429	0.460
Subgraph-level (G4)	0.558	0.641	0.699	0.374	0.396	0.450
Mixed	0.580	0.676	0.757	0.273	0.326	0.389
Best-Single	0.793	0.846	0.892	0.427	0.456	0.503
Ours	**0.813**	**0.874**	**0.909**	**0.885**	**0.903**	**0.916**

These results show that neither committing to one granularity nor directly merging all granularities is sufficient. The gain instead comes from organizing cross-granularity evidence through graph localization, ranking, and coverage-aware selection, which reduce redundant views while preserving complementary structure.

4.4 Why the Proposed Retrieval Pipeline Works

To understand why the proposed pipeline is effective beyond single-granularity and naive mixed retrieval, we next examine two of its key design choices: anchor-based graph localization and later-stage evidence selection.

Table 4 examines the anchor choice used in graph localization. Each variant keeps the same retrieval pipeline and differs only in the anchor granularity. This isolates the effect of anchor choice from the later stages of cross-granularity ranking and coverage-aware evidence selection.

The results show a consistent pattern on both QA and code generation. On QA, G3 achieves 0.8130/0.8737/0.9094 at Recall@5/10/20, outperforming G2, G4, and G1 at all three cutoffs. On code generation, G3 again gives the best results, reaching 0.8849/0.9028/0.9156. These results show that G3 is the most effective anchor granularity in the proposed pipeline. Edge-level units preserve explicit relations while remaining compact enough for reliable graph localization. G2 is the second-best choice because node-level units capture focused object semantics, but they do not represent relations as directly. By contrast, G4 is often too broad to serve as a precise anchor, whereas G1 is often too local to support reliable expansion.

Table 4: Anchor-granularity variants of the proposed pipeline on QA and code generation.

Anchor	QA			CodeGen		
	R@5	R@10	R@20	R@5	R@10	R@20
Property-level (G1)	0.659	0.752	0.796	0.776	0.814	0.836
Node-level (G2)	0.717	0.792	0.838	0.855	0.881	0.893
Edge-level (G3)	**0.813**	**0.874**	**0.909**	**0.885**	**0.903**	**0.916**
Subgraph-level (G4)	0.678	0.752	0.825	0.812	0.856	0.872

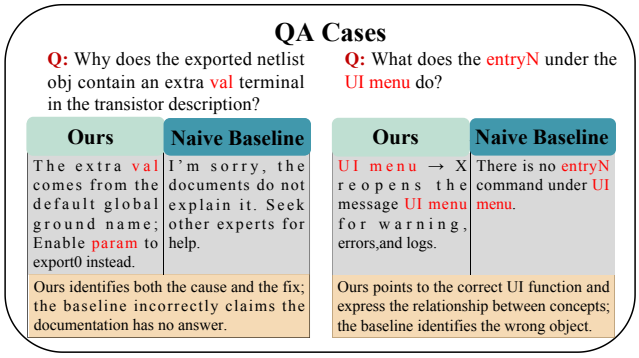


Figure 4: Qualitative end-to-end QA cases of the proposed pipeline.

Table 5 shows that coverage-aware selection is the most critical retained component: replacing it with score-only top-K selection causes the largest consistent drop. Removing the reranker or Multi-ANN also degrades all settings, confirming the need for both diverse candidate entry points and cross-granularity comparison before final selection.

Other components are more task-dependent. One-hop expansion changes Doc results only slightly but reduces GDS CodeGen recall more clearly, while removing BM25 also has a larger effect on GDS. Node Support IDF produces milder gains and mainly stabilizes ranking. Overall, coverage-aware selection, reranking, and multi-entry recall are the most consistent contributors, whereas expansion and lexical or structural corrections depend more on corpus structure.

Figures 4 and 5 compare our method with a naive baseline. Our QA outputs identify the relevant function or both the cause and

Table 5: Selected ablation results on Doc QA, Doc CodeGen, GDS CodeGen, and Combined CodeGen.

Ablation	Doc QA			Doc CodeGen			GDS CodeGen			Combined CodeGen		
	R@5	R@10	R@20	R@5	R@10	R@20	R@5	R@10	R@20	R@5	R@10	R@20
Baseline	**0.813**	**0.874**	0.909	**0.830**	**0.878**	**0.901**	**0.885**	**0.903**	**0.916**	**0.860**	**0.892**	**0.909**
w/o Multi-ANN	0.751	0.854	0.886	0.737	0.836	0.872	0.811	0.841	0.865	0.777	0.839	0.868
1-hop Expansion	0.792	0.874	**0.910**	0.821	0.870	0.891	0.837	0.860	0.869	0.830	0.864	0.879
w/o Reranker	0.711	0.826	0.877	0.737	0.820	0.863	0.837	0.862	0.888	0.792	0.843	0.877
w/o BM25	0.770	0.853	0.904	0.798	0.859	0.899	0.786	0.837	0.863	0.792	0.847	0.880
Score Top-K (w/o MMR)	0.653	0.768	0.853	0.627	0.753	0.828	0.821	0.853	0.883	0.732	0.807	0.857
w/o Node Support IDF	0.753	0.828	0.884	0.809	0.876	0.901	0.846	0.871	0.889	0.818	0.873	0.894

Table 6: KG quality and error attribution.

Dataset	Node Coverage Avg	Edge Coverage Avg	Schema Compliance	Retrieval Miss	KG Miss
Doc	0.998	0.994	1.000	0.206	0.003
GDS	1.000	1.000	1.000	0.600	0.000

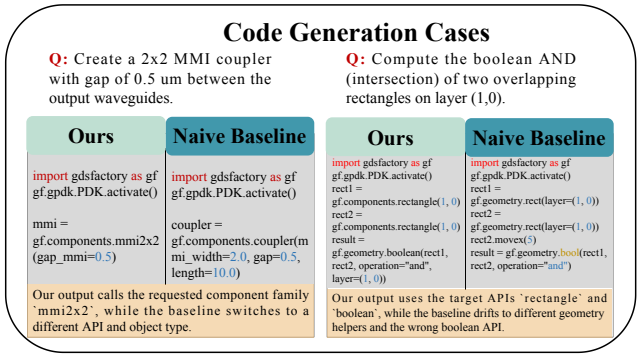


Figure 5: Qualitative end-to-end code generation cases of the proposed pipeline.

fix, whereas the baseline misses the answer or retrieves the wrong object. In code generation, our outputs use the requested APIs while the baseline drifts to different interfaces. The cases therefore connect retrieval gains to grounded answers and API-aligned code.

Beyond API overlap and CodeBLEU on the reproducible GDSFactory benchmark, our method obtains 0.767 executable correctness and a 0.675 pass rate, exceeding the strongest added multi-grained baseline pass rate of 0.642.

4.5 KG Quality and Retrieval Bottlenecks

The constructed Doc QA, Doc CodeGen, and GDS CodeGen KGs contain 11,114/23,115/4,827 nodes, 31,587/45,647/6,934 edges, and 1,648/7,956/761 hyperedges, respectively. These corpus-dependent sizes are reported here as properties of the evaluated graph instances rather than requirements of the general construction method.

Table 6 evaluates whether the constructed graphs preserve benchmark annotations and where the remaining retrieval errors arise. Node and Edge Coverage measure the fractions of annotated graph objects and relations retained after extraction and consolidation, while Schema Compliance checks whether graph instances satisfy the declared type constraints. KG Miss denotes required evidence absent from the graph; Retrieval Miss denotes evidence present in the graph but not recovered by retrieval.

Both graphs achieve near-complete structural coverage and full schema compliance. The small Doc coverage gaps mainly involve implicit aliases and cross-section references, while GDS has no observed KG miss. In contrast, Retrieval Miss is substantially larger than KG Miss for both corpora. Thus, most remaining errors arise after graph construction, when relevant graph evidence must be localized, ranked, and selected under a limited retrieval budget. This attribution identifies evidence recovery, rather than graph completeness, as the primary remaining bottleneck and motivates future improvements to localization and cross-granularity ranking.

5 Conclusion

Our method replaces fixed chunks with schema-grounded evidence at multiple granularities. By localizing query-relevant graph regions and ranking complementary evidence across granularities, it avoids irrelevant coarse context and fragmented procedural dependencies. Consistent QA and code-generation gains across command- and API-centric documentation, especially when evidence spans dependent elements, demonstrate the value of structured retrieval for grounded EDA assistants. This design generalizes across documentation families. It also provides a reusable interface between documentation structure and generation: new documentation families can instantiate their own schemas while retaining the same localization, ranking, and evidence-selection stages, independent of the underlying generator.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (No. 62404021), the Beijing Natural Science Foundation (No. L2603021), the Fundamental Research Funds for the BUPT (No. 2025AI4S20), and the Research Initiation Project for Introduced Talents of BUPT (No. 510124058, 510224062).

References

[1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avi Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *ICLR*.

[2] Jaime G. Carbonell and Jade Goldstein. 1998. The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. In *SIGIR*. 335–336. doi:10.1145/290941.291025

[3] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *CoRR* abs/2402.03216 (2024). doi:10.48550/arXiv.2402.03216

[4] Mark Chen et al. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021).

[5] Zhenghao Chen, Yuan Pu, Hairuo Han, Yuntao Nie, Jiajun Qin, Yuhan Qin, Tairu Qiu, Zhuolun He, Jianwang Zhai, Bei Yu, and Kang Zhao. 2026. MAEDA: An LLM-Powered Multi-Agent Evaluation Framework for EDA Tool Documentation QA. In *2026 Design, Automation & Test in Europe Conference (DATE)*. 1–7. doi:10.23919/DATE69613.2026.11539506

[6] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. 2009. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *SIGIR*. 758–759. doi:10.1145/1571941.1572114

[7] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *CoRR* abs/2404.16130 (2024). doi:10.48550/arXiv.2404.16130

[8] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. LightRAG: Simple and Fast Retrieval-Augmented Generation. *CoRR* abs/2410.05779 (2024).

[9] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *NeurIPS*.

[10] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. In *ICML*, Vol. 119. 3929–3938.

[11] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering. In *NeurIPS*.

[12] Zhuolun He, Yuan Pu, Haoyuan Wu, Yuhan Qin, Tairu Qiu, and Bei Yu. 2025. Large Language Models for EDA: From Assistants to Agents. *Foundations and Trends in Electronic Design Automation* 14, 4 (2025), 295–314. doi:10.1561/1000000063-2

[13] Zhuolun He, Yuan Pu, Haoyuan Wu, Tairu Qiu, and Bei Yu. 2025. Large Language Models for EDA: Future or Mirage? *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 90:1–90:53. doi:10.1145/3736167

[14] Zhuolun He and Bei Yu. 2024. Large Language Models for EDA: Future or Mirage?. In *ISPD*. 65–66. doi:10.1145/3626184.3639700

[15] Cheng-Yu Hsieh, Si-An Chen, Chun-Liang Li, Yasuhisa Fujii, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. 2023. Tool Documentation Enables Zero-Shot Tool-Usage with Large Language Models. *CoRR* abs/2308.00675 (2023). doi:10.48550/arXiv.2308.00675

[16] Yanli Hu, Teng Liu, Zhuangyi Zhou, Weixin Zeng, Zhen Tan, and Xiang Zhao. 2026. Iterative Multi-Granular RAG with Contextual Hierarchical Graph. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 31095–31103. doi:10.1609/aaai.v40i37.40370

[17] Yiqian Huang, Shiqi Zhang, and Xiaokui Xiao. 2025. KET-RAG: A Cost-Efficient Multi-Granular Indexing Framework for Graph-RAG. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*. 1003–1012. doi:10.1145/3711896.3737012

[18] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2022. Unsupervised Dense Information Retrieval with Contrastive Learning. *Transactions on Machine Learning Research* (2022).

[19] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A Survey on Large Language Models for Code Generation. *CoRR* abs/2406.00515 (2024). doi:10.48550/arXiv.2406.00515

[20] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *EMNLP*. 6769–6781. doi:10.18653/V1/2020.EMNLP-MAIN.550

[21] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *SIGIR*. 39–48. doi:10.1145/3397271.3401075

[22] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.

[23] Zhuoqun Li, Xuanang Chen, Haiyang Yu, Hongyu Lin, Yaojie Lu, Qiaoyu Tang, Fei Huang, Xianpei Han, Le Sun, and Yongbin Li. 2025. StructRAG: Boosting Knowledge Intensive Reasoning of LLMs via Inference-time Hybrid Information Structurization. In *ICLR*.

[24] Mingjie Liu, Nathaniel Ross Pinckney, Brucek Khailany, and Haoxing Ren. 2023. Invited Paper: VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *ICCAD*. 1–8. doi:10.1109/ICCAD57390.2023.10323812

[25] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2025. Think-on-Graph 2.0: Deep and Faithful Large Language Model Reasoning with Knowledge-guided Retrieval Augmented Generation. In *ICLR*.

[26] Ghasem Pasandi, Kishor Kunal, Varun Tej, Kunjal Shah, Hanfei Sun, Sumit Jain, Chunhui Li, Chenhui Deng, Teodor-Dumitru Ene, Haoxing Ren, and Sreedhar Pratty. 2025. JARVIS: A Multi-Agent Code Assistant for High-Quality EDA Script Generation. *CoRR* abs/2505.14978 (2025). doi:10.48550/arXiv.2505.14978

[27] Yuan Pu, Zhuolun He, Yuqi Jiang, Tairu Qiu, Haoyuan Wu, Qi Sun, Cheng Zhuo, and Bei Yu. 2025. Customized Retrieval Augmented Generation and Benchmarking for EDA Tool Documentation QA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 12 (2025), 4615–4628. doi:10.1109/TCAD.2025.3568776

[28] Yuan Pu, Zhuolun He, Shutong Lin, Jiajun Qin, Xinyun Zhang, Hairuo Han, Haisheng Zheng, Yuqi Jiang, Cheng Zhuo, Qi Sun, David Z. Pan, and Bei Yu. 2025. MM-GRADE: A Multi-Modal EDA Tool Documentation QA Framework Leveraging Retrieval Augmented Generation. In *ICCAD*. 1–9. doi:10.1109/ICCAD66269.2025.11240857

[29] Yingqi Qu, Yuchen Ding, Jing Liu, Kai Liu, Ruiyang Ren, Wayne Xin Zhao, Daxiang Dong, Hua Wu, and Haifeng Wang. 2021. RocketQA: An Optimized Training Approach to Dense Passage Retrieval for Open-Domain Question Answering. In *NAACL-HLT*. 5835–5847. doi:10.18653/V1/2021.NAACL-MAIN.466

[30] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. *CoRR* abs/2009.10297 (2020).

[31] Stephen E. Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389. doi:10.1561/1500000019

[32] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. In *NAACL-HLT*. 3715–3734. doi:10.18653/V1/2022.NAACL-MAIN.272

[33] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In *ICLR*.

[34] Haitian Sun, Tania Bedrax-Weiss, and William W. Cohen. 2019. PullNet: Open Domain Question Answering with Iterative Retrieval on Knowledge Bases and Text. In *EMNLP-IJCNLP*. 2380–2390. doi:10.18653/V1/D19-1242

[35] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *ICLR*.

[36] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024), 46:1–46:31. doi:10.1145/3643681

[37] Ning Wang, Bingkun Yao, Jie Zhou, Xi Wang, Zhe Jiang, and Nan Guan. 2024. Large Language Model for Verilog Generation with Code-Structure-Guided Reinforcement Learning. *CoRR* abs/2407.18271 (2024). doi:10.48550/arXiv.2407.18271

[38] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In *EMNLP*. 1069–1088. doi:10.18653/V1/2023.EMNLP-MAIN.68

[39] Haoyuan Wu, Zhuolun He, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. 2024. ChatEDA: A Large Language Model Powered Autonomous Agent for EDA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 10 (2024), 3184–3197. doi:10.1109/TCAD.2024.3383347

[40] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021. QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering. In *NAACL-HLT*. 535–546. doi:10.18653/V1/2021.NAACL-MAIN.45

[41] Donghan Yu, Chenguang Zhu, Yuwei Fang, Wenhao Yu, Shuohang Wang, Yichong Xu, Xiang Ren, Yiming Yang, and Michael Zeng. 2022. KG-FiD: Infusing Knowledge Graph in Fusion-in-Decoder for Open-Domain Question Answering. In *ACL*. 4961–4974. doi:10.18653/V1/2022.ACL-LONG.340

[42] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *EMNLP*. 2471–2484. doi:10.18653/V1/2023.EMNLP-MAIN.151

[43] Shuyan Zhou, Uri Alon, Frank F. Xu, Zhengbao Jiang, and Graham Neubig. 2023. DocPrompting: Generating Code by Retrieving the Docs. In *ICLR*.